



Micro-Agents — Read-Side v1

Architecture & Infrastructure Design · Mycelium Framework

VibeSpace LLC · Space Cowboy #9 · 2026-05-29 · Patent-pending · companion to docs/superpowers/specs/2026-05-29-micro-agents-readside-v1-design.md

READ-SIDE

AUDIT-RUN ONLY

NO CULTIVATE.TS CHANGE

NO NEW DEPS

MEASURE-TO-GATE-V2

1 · What this is, in one paragraph

PR #3 shipped micro-agents as **decorative state objects** — 2–4 spheres orbiting each leaf on the dashboard, doing no real work. The cache-network beneath them **is** real and wired. This design makes micros real as **read-side context-gatherers** inside the audit-run testers-pool: cheap (Haiku) read-only sub-sessions summarize a tester's input targets, those summaries fold into the tester's prompt, and the expensive (Opus) tester reads the compact summary instead of re-reading raw sources. Every micro call rides the **existing** content-addressed cache, so identical sub-jobs across testers are free. The whole thing is instrumented so its first real run answers one question: **does the cost model actually pay?** — which gates the more ambitious write-side v2.

The one-line invariant. v1 saves money iff, across the pool with cache hits counted:

$\Sigma(\text{folded Opus input} + \text{Haiku input}) < \Sigma(\text{baseline raw Opus input})$.

Input tokens only (near-deterministic) → a single before/after diff is honest, no N-run statistics.

2 · System architecture

```
audit-run testers-pool      (cli/src/lib/audit/testers-pool.ts – existing fan-out)
  |
  | for each tester, BEFORE its main assertion run:
  |
  | runMicroFanout(tester, ctx)      NEW cli/src/lib/micro-agents/fanout.ts
  |
  | 1. derivePlan(tester)            NEW pure · NO LLM
  |    tester.INPUTS → 2–4 MicroSubJob[] (capped by xorshift seeded count)
  |
  | 2. worker micros (2–4, concurrent via runWithConcurrency)
  |    each: query({ model:'haiku', tools:[Read,Grep,Glob], maxTurns:3 })
  |    cache-checked individually: store.get(cacheKey) → FREE on cross-tester hit
  |    output: one compact summary of one input target      (read-only → no writes)
  |
  | 3. fold: summaries → "## Gathered Context" prepended to tester prompt
  |
  | tester main query()              (testers-runner.ts:286 – model UNCHANGED = Opus-class)
  |   step-1 instruction flipped: "rely on Gathered Context; re-READ raw only if insufficient"
  |   → consumes compact folded context instead of re-reading raw sources ← THE SAVING
  |   → emits findings JSONL exactly as today      (assertion contract untouched)
```

Reused verbatim (zero contract change)

- `cacheKey()` — sha256 content-addressed key
- `makeCacheStore()` — LRU store + events

Verified mechanism facts

- SDK exposes `model:'haiku' | 'opus' | ...` (coreTypes.d.ts:347) — no fork

- `runWithConcurrency()` — bounded fan-out
- `cache.pulse` aggregator (1.4s) → `state.json`
- `SDK query().activeSessions` shutdown set
- Testers run on **default Opus-class** model (no `model` set @ :286) → real offload exists
- Read-only tools → **zero write-race** (no `run8 #2/#3` exposure)
- `derivePlan` pure → deterministic, testable

3 • Data contracts

```

interface MicroSubJob {
  id: string;          // stable slug
  kind: "read"|"grep"|"summarize";
  target: string;      // from INPUTS
  question: string;    // fixed tpl
}

interface MicroCallRecord {
  tester_id: string;
  sub_job_id: string;
  cache_status: "hit" | "miss";
  model: "haiku";
  input_tokens: number;    // 0 on hit
  output_tokens: number;   // logged, not headlined
  raw_target_bytes: number; // source the summary replaces
}

```

All additive to NUTRIENTS as a new `micro_call` event kind + these shapes. No existing field removed or renamed. `MicroState` gains optional real fields so the dashboard spheres reflect real work.

4 • Measurement & the v2 decision gate

v1's product is **evidence**, not savings. Two risks are promoted to first-class metrics: compression might be nominal (**Risk 1**); testers might re-read raw anyway (**Risk 2**).

METRIC	FORMULA	TELLS US
Net input-token delta (headline)	$\Sigma(\text{folded 0pus in} + \text{Haiku in}) - \Sigma(\text{baseline raw 0pus in})$	Did v1 pay?
Dedup rate	cache hits / total micro calls	Cross-tester reuse
Re-read rate	redundant raw Reads / folded targets	Why it paid or didn't (Risk 2)
Compression ratio	$\Sigma \text{ summary tokens} / \Sigma \text{ raw bytes}$	Summaries real or nominal (Risk 1)

Baseline via a new `--no-micros` flag (additive, mirrors `--no-cache`): run once off, once on, diff `micro-metrics.json`.

OUTCOME	VERDICT
Net delta < 0 and re-read rate low	Read-side pays → scope write-side v2
Net delta ≥ 0 but re-read rate high	Sabotaged by re-reads, not disproven → fix steering, re-measure
Net delta ≥ 0 and low re-read and poor compression	Doesn't pay here → do NOT build v2 (cheap kill signal)

Known limitation. The audit pool proves read-side is *safe* and measures *dedup* well, but testers do precise assertion work that resists summarization — so the pool may *under-show compression* vs. broad-synthesis cultivate leaves. A weak compression number here means "audit testers are the hardest case," not "read-side fails." The re-read metric disambiguates.

5 • Failure isolation

Governing rule: **a micro failure never fails or degrades the tester** — it falls back to baseline (reads raw itself).

FAILURE	HANDLING	EVENT
Worker throws / times out	Drop summary; tester re-reads that raw target	micro.work_failed
Empty / garbage summary	Treated as no-summary (no quality check in v1 — v2's reviewer)	micro.work_empty
derivePlan → 0 sub-jobs	Skip fan-out; tester runs as today; zero overhead	micro.plan_empty
Over USD cap	Micros draw from tester's existing maxBudgetUsd; skip if exceeding	micro.budget_skipped

Structural payoff of read-side: read-only tools → a failed/interrupted micro leaves zero artifacts and zero git state. Nothing to roll back.

6 • Infra footprint & rule compliance

New files

cli/src/lib/micro-agents/	
derive-plan.ts	NEW pure
fanout.ts	NEW orchestrator
metrics.ts	NEW pure
types.ts	NEW
spawn.ts / registry.ts	EDIT additive
derive-plan.test.ts	NEW vitest
metrics.test.ts	NEW vitest

Edited (audit-run only)

cli/src/lib/audit/	
testers-runner.ts	EDIT fanout + fold
testers-pool.ts	EDIT --no-micros + metrics
cli/src/commands/	
audit-run.ts	EDIT register flag
cli/src/commands/cultivate.ts	UNTOUCHED

RULE	STATUS
#2 cultivate.ts contract	✓ untouched
#4 no new deps	✓ reuses SDK + cache + concurrency
#5 framework tests	⚠ pure fns only (derive-plan, metrics) — security/ precedent
#9 state.json shape	✓ additive optional MicroState fields
#10 Upgrade interface	✓ untouched

Mycelium • VibeSpace LLC • "The network provides." • Read-side v1 is the instrument; the write-side thin-LLM coding agent (docs/micro-agents-vision.md) is the destination.